

Back-End Technologies

BIS605

Software Development Technologies for Digital Business

Asst. Prof. Dr. Worarat Krathu

Resources

- <https://www.php.net/>
- Taha Malampatti, PHP: Hypertext Preprocessor, at <https://www.slideshare.net/>
- Vibrant Technologies & Computers, PHP Training in Mumbai, at <https://www.slideshare.net/>
- Bhargav Anadkat, Basic Concept of Node.js & NPM, at <https://www.slideshare.net/>
- BG Java EE Course, Java Servlets, at <https://www.slideshare.net/>
- Shivam Gupta, Python, at <https://www.slideshare.net/>
- <https://www.w3schools.com/php>
- <https://www.educba.com/jsp-vs-php/>
- <https://www.tutorialspoint.com/servlets/servlets-first-example.htm>
- <https://www.educative.io/blog/web-development-in-python>
- <https://docs.python.org/3/tutorial/index.html>

NODE.JS



Node.JS

- An open source, cross platform runtime environment for server side and networking application.
- Basically, NodeJS is server side programming language like PHP, C#, Python etc.
- It is 40% JS and 60% C++.
- It provided an event driven architecture and a non blocking I/O that optimize and scalability. These technology uses for real time application
- ***‘Node's goal is to provide an easy way to build scalable network programs’ - (from nodejs.org!)***

Node.JS

- It used Google JavaScript V8 Engine to Execute Code.
- **V8** is an open source JavaScript engine developed by Google. Its written in C++ and is used in Google Chrome Browser.
- It is invented in 2009 by Ryan Dahl.
- It is popular in development because front & backend side both uses JavaScript Code.

Node.JS Non Blocking I/O

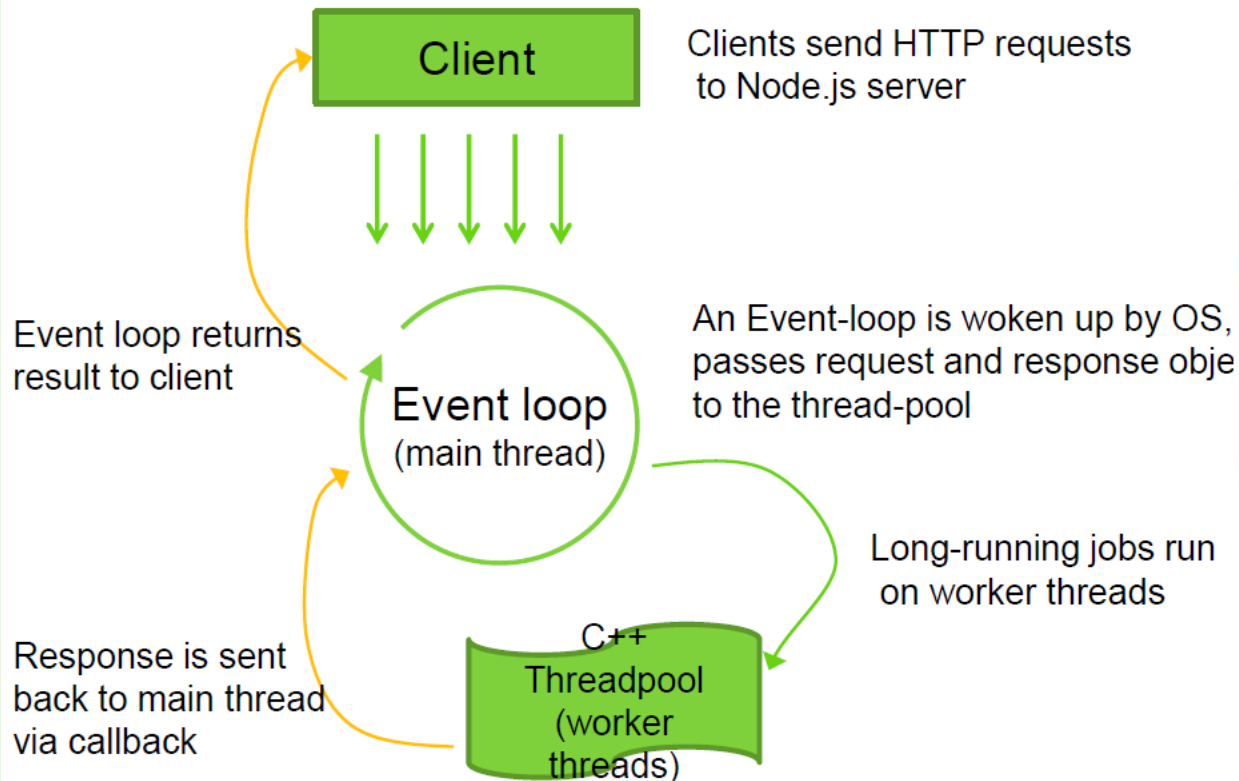
- Traditional I/O

```
var result = db.query("select x from table_Y");  
doSomethingWith(result); //wait for result!  
doSomethingWithoutResult(); //execution is blocked!
```

- Non-traditional, Non-blocking I/O

```
db.query("select x from table_Y",function (result){  
    doSomethingWith(result); //wait| for result!  
});  
doSomethingWithoutResult(); //executes without any  
delay!
```

Node.JS



JavaScript		C/C++		
node standard library				
node bindings (socket, http, etc)				
V8	thread pool (libeio)	event loop (libev)	DNS (c-ares)	crypto (OpenSSL)

Node.JS

- Programs for Node.js are written in JavaScript but not in the same JavaScript we are use to. There is no DOM implementation provided by Node.js, i.e. you **can not** do this:
 - `var element = document.getElementById("elementId");`
- Node Package Manager (NPM) is the package manager of Node.js and JavaScript where we can create public and private module
- Find all modules on NPM website : <https://www.npmjs.com/>

Node.JS

- Some good modules
 - **Express** – to make things simpler e.g. syntax, DB connections.
 - **Jade** – HTML template system
 - **Socket.IO** – to create real-time apps
 - **Nodemon** – to monitor Node.js and push change automatically
 - **CoffeeScript** – for easier JavaScript development

JAVA (JSP / SERVLET)



Java Servlet

- Technology for generating dynamic Web pages (like PHP, ASP, ASP.NET, ...)
- Protocol and platform-independent server side components, written in Java, which extend the standard Web servers
- Java programs that serve HTTP requests

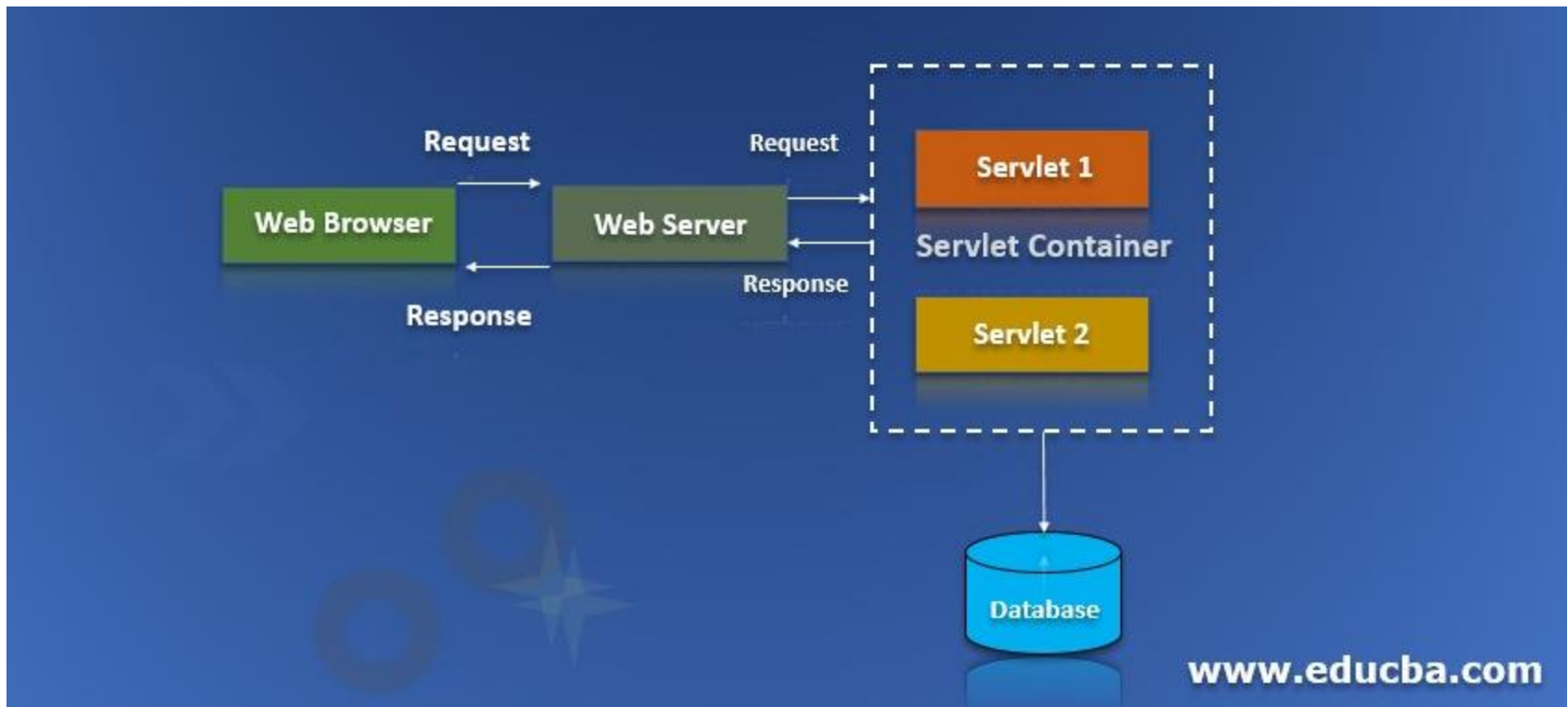
JSP

- JSP is a server-side programming technology that helps in creating dynamic web pages.
- JSP is an abstraction of Servlets, which is based on the Java programming language.
- JSPs are translated into Servlets at runtime; thus, every JSP can be called a Servlet in disguise.

JSP

- JSP enables developers to insert Java code into HTML pages by using special tags and standard actions.
- A developer can also create custom tags built on custom logic.
- Thus, JSPs are very flexible when it comes to performing logical flows.

Servlet Architecture



Why Java Servlet

- Portability
 - Write once, serve everywhere
- Power
 - Can take advantage of all Java APIs
- Elegance
 - Simplicity due to abstraction
- Efficiency & Endurance
 - Highly scalable
- Safety
 - Strong type-checking
 - Memory management

Why Java Servlet

- Integration
 - Servlets tightly coupled with server
- Extensibility & Flexibility
 - Servlets designed to be easily extensible, though currently optimized for HTTP uses
 - Flexible invocation of servlet (SSI, servlet-chaining, filters, etc.)

Example of Servlet

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloWorld extends HttpServlet {

    private String message;

    public void init() throws ServletException {
        // Do required initialization
        message = "Hello World";
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Set response content type
        response.setContentType("text/html");

        // Actual logic goes here.
        PrintWriter out = response.getWriter();
        out.println("<h1>" + message + "</h1>");
    }

    public void destroy() {
        // do nothing.
    }
}
```

```
<servlet>
    <servlet-name>HelloWorld</servlet-name>
    <servlet-class>HelloWorld</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>HelloWorld</servlet-name>
    <url-pattern>/HelloWorld</url-pattern>
</servlet-mapping>
```



PYTHON



Python

- Python is an easy to learn, powerful programming language.
- It has efficient high-level data structures and a simple but effective approach to object-oriented programming.
- The Python interpreter and the extensive standard library are freely available from the Python Web site

Python Example

```
x = 5  
y = "John"  
print(x)  
print(y)
```

```
5  
John
```

```
i = 1  
while i < 6:  
    print(i)  
    i += 1
```

```
1  
2  
3  
4  
5
```

```
x = 4  
x = "Sally"  
print(x)
```

```
Sally
```

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

```
apple  
banana  
cherry
```

Python for Web Development

- Easy to learn
 - The language relies on common expressions and whitespace, which allows you to write significantly less code compared to some other languages like Java or C++.
- Rich ecosystem and libraries
 - Python offers a vast range of library tools and packages, which allows you to access much pre-written code, streamlining your application development time.
- Fast prototyping
 - Because Python takes significantly less time to build your projects compared to other programming languages, your ideas come to life a lot faster

Django

- A Python web framework that offers an open-source, high-level framework.
- It's fast, secure, and scalable.
- Django offers strong community support and detailed documentation.

Flask

- A microframework, which is a minimalistic web framework.
- It's less “batteries-included,” meaning that it lacks a lot of features and functionality that full-stack frameworks like Django offer.
- The philosophy behind Flask is that it gives only the components you need to build an app so that you have the flexibility and control.

Python Libraries

- Web development
 - Scrapy - web crawler to extract data for your application
 - Zappa - developing a serverless application on AWS Lambda
 - Requests - a library that allows you to send HTTP requests easily, which is used to communicate with an application
 - Dash - data visualization which built on top of Flask
- General application development
 - Collections – Container datatypes
 - CSV – file handling
 - Numpy - machine learning libraries
 - TensorFlow - computational library for writing new algorithms that involve a large number of tensor operations
 - Scikit-learn – libraries for working with complex data.
 - Keras - machine learning libraries for neural networks

PHP



PHP: Hypertext Preprocessor

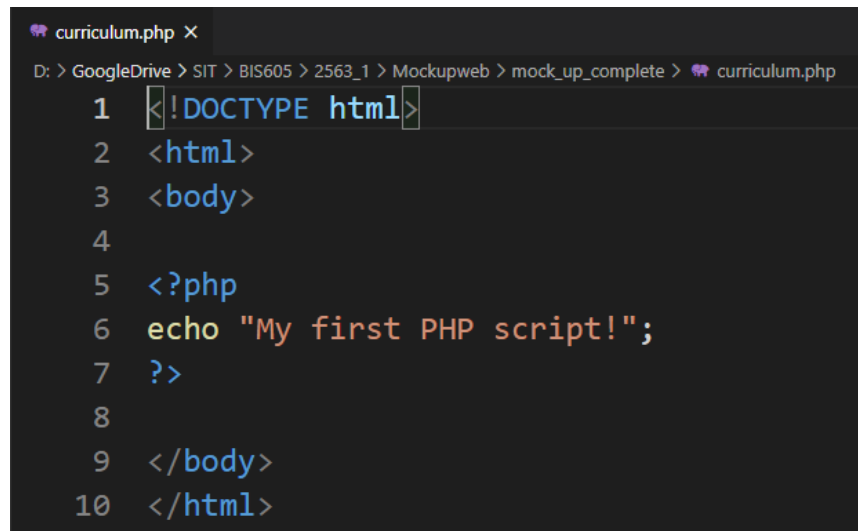
- PHP is a popular general-purpose scripting language that is especially suited to web development
- It is similar to JavaScript but its scripts are executed on server side
 - PHP code is embedded in HTML using tags
 - when a page request arrives, the server recognizes PHP content via the file extension (*.php* or *.phtml*)
 - the server executes the PHP code, substitutes output into the HTML page
 - the resulting page is then downloaded to the client
 - user never sees the PHP code, only the output in the page
- It is free to download and use

PHP

- PHP runs on different platforms (Windows, Linux, Unix, Mac OS X, etc.)
- PHP is compatible with almost all servers used today (Apache, IIS, etc.)
- PHP has support for a wide range of databases
- PHP is free. Download it from the official PHP resource: www.php.net
- PHP is easy to learn and runs efficiently on the server side

PHP File

- PHP files can contain text, HTML, JavaScript code, and PHP code
- PHP code are executed on the server, and the result is returned to the browser as plain HTML
- PHP files have a default file extension of ".php"



```
curriculum.php X
D: > GoogleDrive > SIT > BIS605 > 2563_1 > Mockupweb > mock_up_complete > curriculum.php
1  <!DOCTYPE html>
2  <html>
3  <body>
4
5  <?php
6  echo "My first PHP script!";
7  ?>
8
9  </body>
10 </html>
```

PHP

- PHP can generate dynamic page content
- PHP can create, open, read, write, and close files on the server
- PHP can collect form data
- PHP can send and receive cookies
- PHP can add, delete, modify data in your database
- PHP can restrict users to access some pages on your website
- PHP can encrypt data

Basic PHP Syntax

- A PHP script can be placed anywhere in the document. A PHP script starts with **<?php** and ends with **?>**
- The default file extension for PHP files is ".php".

```
curriculum.php X
D: > GoogleDrive > SIT > BIS605 > 2563_1 > Mockupweb > mock_up_complete > curriculum.php
1  <!DOCTYPE html>
2  <html>
3  <body>
4
5  <?php
6  echo "My first PHP script!";
7  ?>
8
9  </body>
10 </html>
```

Basic PHP Syntax

- A PHP scripting block can be placed (almost) anywhere in an HTML document.
- *print* and *echo* for output
- a semicolon (;) at the end of each statement
- // for a single-line comment
- /* and */ for a large comment block.

```
<html>
<!-- hello.php -->
<head><title>Hello World</title></head>
<body>
  <p>This is going to be ignored by the PHP interpreter.</p>
  <?php echo '<p>While this is going to be parsed.</p>'; ?>
  <p>This will also be ignored by the PHP preprocessor.</p>
  <?php print('<p>Hello and welcome to <i>my</i> page!</p>');
  ?>
  <?php
    //This is a comment
    /*
     This is
     a comment
     block
    */
  ?>
</body>
</html>
```

Basic PHP Syntax

- The PHP command '*echo*' is used to output the parameters passed to it
- The typical usage for this is to send data to the client's web-browser

```
curriculum.php X
D: > GoogleDrive > SIT > BIS605 > 2563_1 > Mockupweb > mock_up_complete > curriculum.php
1  <!DOCTYPE html>
2  <html>
3  <body>
4
5  <?php
6  echo "My first PHP script!";
7  ?>
8
9  </body>
10 </html>
```

PHP Variables

- PHP is a loosely typed language
 - we do not have to tell PHP which data type the variable is. PHP automatically converts the variable to the correct data type, depending on its value.
- PHP variables must begin with a “\$” sign
- Case-sensitive (\$Foo != \$foo != \$fOo)
- Global and locally-scoped variables
 - Global variables can be used anywhere
 - Local variables restricted to a function or class
- Certain variable names reserved by PHP
 - Form variables (\$_POST, \$_GET)
 - Server variables (\$_SERVER)
 - Etc.

Operators

- **Arithmetic Operators:** $+$, $-$, $*$, $/$, $\%$, $++$, $--$
- **Assignment Operators:** $=$, $+=$, $-=$, $*=$, $/=$, $\%=$
- **Comparison Operators:** $==$, $!=$, $>$, $<$, $>=$, $<=$
- **Logical Operators:** $\&\&$, $||$, $!$
- **String Operators:** $.$ and $.=$ (for string concatenation)

Example	Is the same as
$x+=y$	$x=x+y$
$x-=y$	$x=x-y$
$x*=y$	$x=x*y$
$x/=y$	$x=x/y$
$x\%=y$	$x=x\%y$

```
$a = "Hello ";  
$b = $a . "World!"; // now $b contains "Hello World!"  
$a = "Hello ";  
$a .= "World!";
```

Variable Usage

```
<?php
$foo = 25;           // Numerical variable
$bar = "Hello";      // String variable

$foo = ($foo * 7);    // Multiplies foo by 7
$bar = ($bar * 7);    // Invalid expression
?>
```

Control Flow

- If...Else...

```
<?php
If ($user=="John")
{
    Print "Hello John.";
}
Else
{
    Print "You are not John.";
}
?>
```

Control Flow

- While loop

```
<?php
$count=0;
While ($count<3)
{
    Print "hello PHP. ";
    $count += 1;
    // $count = $count + 1;
    // or
    // $count++;
?>
```

- This will print:
 - *hello PHP. hello PHP. hello PHP.*

Function

- Functions MUST be defined before then can be called

```
<?php
    // This is a function
    function foo($arg_1, $arg_2)
    {
        $arg_2 = $arg_1 * $arg_2;
        return $arg_2;
    }

    $result_1 = foo(12, 3);           // Store the function
    echo $result_1;                   // Outputs 36
    echo foo(12, 3);                  // Outputs 36
?>
```

Form

- Access to the HTTP POST and GET data is simple in PHP
- The global variables `$_POST[]` and `$_GET[]` contain the request data
- Form can indicate the method to submit data, post or get
- When getting data from the form, in PHP you must indicating the method and variable name

```
<body>
  <h3>Please select your interested PhD
  Curriculum</h3>
  <form action="" method="post">
```

```
<?php
include 'model.php';
if(isset($_POST["SubmitButton"])) {
```

Form

- GET Vs POST

	GET	POST
BACK button/Reload	Harmless	Data will be re-submitted (the browser should alert the user that the data are about to be re-submitted)
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Not cached
Encoding type	application/x-www-form-urlencoded	application/x-www-form-urlencoded or multipart/form-data. Use multipart encoding for binary data
History	Parameters remain in browser history	Parameters are not saved in browser history
Restrictions on data length	Yes, when sending data, the GET method adds the data to the URL; and the length of a URL is limited (maximum URL length is 2048 characters)	No restrictions
Restrictions on data type	Only ASCII characters allowed	No restrictions. Binary data is also allowed
Security	GET is less secure compared to POST because data sent is part of the URL Never use GET when sending passwords or other sensitive information!	POST is a little safer than GET because the parameters are not stored in browser history or in web server logs
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

Exercise 10.1

- Create a webpage to calculate BMI which takes input of weight and height from user and display BMI as well as image representing if a user is thin, healthy or fat.

Exercise 10.2

- Create a webpage to calculate a simple expression consisting of 2 operands and 1 operator.
 - 1 input of user name
 - 2 input of number
 - 1 input of operator (+ , - , * , /)
 - Output must display user name and result of calculation.